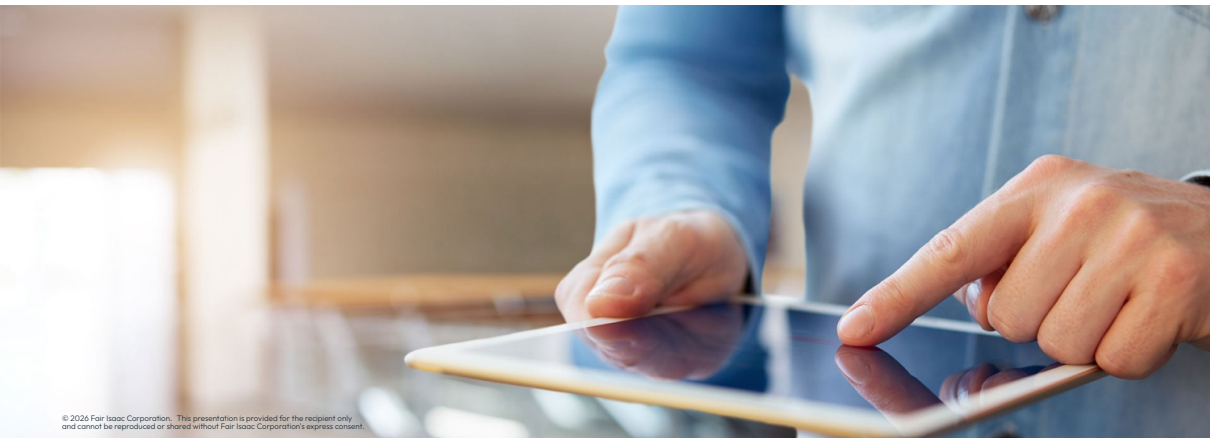




Solving and Querying





Solving and querying a
problem

Solving a problem

- The method `problem.optimize()` is used to solve an optimization problem that was either built via Python functions or read from a file:

```
p.optimize(flag)
```

- The algorithm is determined automatically as follows:
 - If all **variables are continuous**, the problem is solved as a continuous optimization problem
 - If **at least one integer variable** was declared, then the problem will be solved as a mixed integer (linear, quadratically constrained, or nonlinear) problem
 - If the problem contains **nonlinear constraints that are non-quadratic and non-conic**, then the appropriate nonlinear solver of the FICO® Xpress Optimization suite will be called: either Xpress Global or Xpress NonLinear, depending on available licenses



Note: *Non-convex quadratic problems are included in the base offering of the FICO® Xpress Solver license and will by default be solved with the Xpress Global technology*

Solve and solution status

- The *solve* and *solution* statuses of a problem can be obtained via the *solvestatus* and *solstatus* attributes using `problem.attributes.<attribute>`, which are also returned by the `problem.optimize()` function:

```
solvestatus, solstatus = p.optimize()
```

- Where the value of:
 - *solvestatus* can be {COMPLETED, STOPPED, FAILED, UNSTARTED}
 - *solstatus* can be {FEASIBLE, OPTIMAL, INFEASIBLE, UNBOUNDED, NOTFOUND}
- The statuses can then be conveniently queried as follows:

```
if solvestatus == xp.SolveStatus.COMPLETED:  
    print("Solve completed with solution status: ", solstatus.name)  
else:  
    print("Solve status: ", solvestatus.name)
```

Querying a problem

- The method `problem.getSolution()` returns the optimal solution as a list:
 - An argument can be passed in the form of a list, dictionary, tuple, or any sequence (including *NumPy* arrays) of `variables`, `indices`, `strings`, `expressions` and other aggregate objects
 - If an optimal solution was not found but at least one feasible solution is available, data based on the best feasible solution will be returned

Querying a problem

- The method `problem.getSolution()` returns the optimal solution as a list:
 - An argument can be passed in the form of a list, dictionary, tuple, or any sequence (including *NumPy* arrays) of **variables, indices, strings, expressions** and other aggregate objects
 - If an optimal solution was not found but at least one feasible solution is available, data based on the best feasible solution will be returned
- Examples:

```
p.optimize()
```

```
print(p.getSolution())
```

prints a list with an optimal solution

```
print("v1 is", p.getSolution(v1))
```

only prints the value of v1

```
a = p.getSolution(x)
```

gets the values of all variables in the list x

```
b = p.getSolution(range(4))
```

gets the value of the first four variables

```
c = p.getSolution('Var1')
```

gets the value of a variable by its name

```
d = p.getSolution(v1 + 3*x)
```

gets the value of an expression for the solution

```
e = p.getSolution(np.array(x))
```

gets a NumPy array with the solution of x

Querying a problem

- The method `problem.getSlacks()` retrieves the slack for one or more constraints of the problem w.r.t. the solution found:
 - Works with indices, constraint names, constraint objects, and lists thereof

```
print(p.getSlacks())           # prints a list of slacks for all constraints
print("slack_1 is", p.getSlacks(cons1)) # only prints the slack of cons1

a = p.getSlacks(conlist)      # gets the slacks of all constraints in list 'conlist'
b = p.getSlacks(range(2))     # gets the slacks of the first 2 constraints of the problem
```



Note: Both methods `p.getSolution()` and `p.getSlacks()` work for continuous or mixed integer problems

Querying a problem

- For problems that only have continuous variables, the two methods `problem.getDUALS()` and `problem.getRCosts()` return the list of dual variables and reduced costs, respectively:
 - Their usage is similar to that of `problem.getSlacks()`

```
print("Duals of last two constraints:", p.getDUALS(constr[-2:]))  
print("Reduced costs of first two variables:", p.getRCosts(x[:2]))
```

Querying a problem

- For problems that only have continuous variables, the two methods `problem.getDuals()` and `problem.getRCosts()` return the list of dual variables and reduced costs, respectively:

- Their usage is similar to that of `problem.getSlacks()`

```
print("Duals of last two constraints:", p.getDuals(constr[-2:]))  
print("Reduced costs of first two variables:", p.getRCosts(x[:2]))
```

- The inner workings of the Python interface obtain a copy of the whole solution, slack, dual, or reduced cost vectors, even if only one element is requested:
 - Instead of repeated calls to `p.getSolution()` or `p.getSlacks()`, it is advisable to make one call and store the result in a list to be consulted in a loop:

```
sol = p.getSolution()  
for i in N:  
    if sol[i] > 1e-3:  
        print(i)
```



www.fico.com/optimization